

Binary Search: Invariants, Boundaries, and Off-by-One Errors

Original source written for the Museion Build Week demonstration.

Version: 1.0

Intended audience: learners who can compare integers and read array indices.

1. The problem binary search solves

Binary search looks for a target in a sequence that is already sorted. It avoids checking every element. Instead, it keeps an **active interval** of indices where the target could still be located.

For an array `values`, let `low` and `high` be inclusive indices. At the start, the active interval is usually the whole array:

```
low = 0
high = values.length - 1
```

The central promise of the algorithm is not merely that the interval becomes smaller. The promise is an **invariant**:

If the target occurs in the array, at least one occurrence remains inside the active interval from `low` through `high`.

An invariant is a statement that is true before an iteration, remains true after the iteration, and helps explain why the algorithm is correct. In binary search, every update must preserve the possibility of finding the target while discarding positions that have been proven impossible.

The loop continues while the interval is nonempty:

```
while low <= high
```

When `low > high`, no candidate index remains. At that point, the target is absent.

2. Choosing and inspecting the midpoint

During each iteration, binary search chooses an index near the center of the active interval. A common midpoint rule is:

```
mid = low + floor((high - low) / 2)
```

This expression chooses the lower middle index when the interval contains an even number of elements. It is also preferable to $(low + high) / 2$ in languages where adding two large indices could overflow.

After choosing `mid`, compare `values[mid]` with the target.

There are three cases:

1. `values[mid] == target`: the search has succeeded.
2. `values[mid] < target`: every index at or below `mid` can be discarded, because the sequence is sorted.
3. `values[mid] > target`: every index at or above `mid` can be discarded.

The midpoint is not a vague suggestion about where to search next. The comparison gives a proof that one whole region cannot contain the target.

Example:

```
values = [2, 5, 8, 11, 14, 19, 23]
target = 19
low = 0
high = 6
mid = 3
values[mid] = 11
```

Because $11 < 19$, indices 0 through 3 are impossible. The next active interval must begin at index 4.

3. Correct boundary updates

When `values[mid] < target`, use:

```
low = mid + 1
```

When `values[mid] > target`, use:

```
high = mid - 1
```

The `+ 1` and `- 1` are essential. The comparison has already shown that index `mid` is not the target. Keeping `mid` inside the next interval would preserve a candidate that has been disproved.

These updates preserve the invariant:

- If the middle value is too small, a target can only appear strictly to the right of `mid`.
- If the middle value is too large, a target can only appear strictly to the left of `mid`.

They also guarantee progress. Whenever the target is not found, the number of candidate indices decreases.

Continuing the earlier example:

```
values = [2, 5, 8, 11, 14, 19, 23]
target = 19
```

```
iteration 1: low = 0, high = 6, mid = 3, value = 11
update: low = 4
```

```
iteration 2: low = 4, high = 6, mid = 5, value = 19
result: found at index 5
```

The interval changed from seven candidates to three candidates, then the target was found.

4. The off-by-one misconception

A common error is to write `high = mid` when the middle value is too large, or `low = mid` when the middle value is too small.

Consider this search:

```
values = [2, 5]
target = 2
low = 0
high = 1
mid = 0
values[mid] = 2
```

This case succeeds immediately. Now consider a target that is absent:

```
values = [2, 5]
target = 4
low = 0
high = 1
mid = 0
values[mid] = 2
```

Because $2 < 4$, the correct update is `low = 1`. If an implementation uses `low = mid`, then `low` remains 0. The next iteration chooses `mid = 0` again. The state does not change, so the loop may never terminate.

The misconception behind this bug is usually:

The learner treats `mid` as still uncertain even after the comparison has ruled it out.

A useful counterexample therefore uses a tiny interval where reusing `mid` visibly repeats the same state. The correction is conceptual, not merely syntactic: once the middle value is known to be too small or too large, the middle index is no longer a candidate.

5. Empty intervals and termination

The inclusive-boundary version of binary search uses the loop condition:

```
while low <= high
```

A one-element interval has `low == high`. That element still needs to be inspected. The interval becomes empty only when `low > high`.

Example with an absent target:

```
values = [2, 5, 8, 11, 14]
target = 6
```

```
low = 0, high = 4, mid = 2, value = 8
value is too large, so high = 1
```

```
low = 0, high = 1, mid = 0, value = 2
value is too small, so low = 1
```

```
low = 1, high = 1, mid = 1, value = 5
value is too small, so low = 2
```

```
now low = 2 and high = 1
the interval is empty, so the target is absent
```

Termination follows from two facts:

1. `low` never decreases and `high` never increases.
2. On every unsuccessful iteration, either `low` increases beyond `mid` or `high` decreases below `mid`.

Therefore the active interval strictly shrinks until the target is found or no candidates remain.

6. A reasoning checklist and transfer case

When tracing binary search, ask these questions in order:

1. What indices are currently inside the active interval?
2. Which midpoint rule is being used?
3. What does the comparison prove impossible?
4. Which boundary must move past `mid`?
5. Is the invariant still true after the update?
6. Did the interval strictly shrink?

Reference pseudocode:

```
function binarySearch(values, target):
    low = 0
    high = values.length - 1

    while low <= high:
        mid = low + floor((high - low) / 2)

        if values[mid] == target:
            return mid
        else if values[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return not-found
```

Transfer case:

```
values = [1, 4, 7, 9, 13, 18, 21, 26, 31]
target = 13
```

To demonstrate understanding, predict each (`low`, `high`, `mid`) state without hints, explain why the discarded region is impossible, and identify the final index. The surface values are new, but the invariant and boundary logic are unchanged.